

For example 1: If the initial queue with size=5 is empty and do with the following instructions : enqueue(5), enqueue(10), dequeue(), enqueue(15), dequeue()., what value will remain on the queue?

Answer: **[15]** (ค่าที่เหลืออยู่ใน Queue คือ [15], Front=15, Back=15)

```
* Trace Ex 1: enq(5) [5] -> enq(10) [5, 10] -> deq() (5 ออก) [10] -> enq(15) [10, 15] -> deq() (10 ออก) -> เหลือ [15]
```

For example 2: If the exist queue with size=5 contains the numbers [20, 30, 40] , and do with the following instructions : dequeue(), enqueue(50), dequeue(), enqueue(60). , what value will remain on the queue?

Answer: **[40, 50, 60]** (ค่าคงเหลือใน Queue, Front=40, Back=60)

```
* Trace Ex 2: เริ่มต้น [20, 30, 40] -> deq() (20 ออก) -> enq(50) -> deq() (30 ออก) -> enq(60) -> เหลือ [40, 50, 60]
```

For example 3: if there is no queue's instance variable before, write statements in main program to create the instance variable myqueue with size=5. Then enqueue the number 1, 2, 3, 4, 5 in that order, and dequeue 3 times by using class Queue's methods that we learned. Finally write the element that remain on the queue's instance variable myqueue.

[เฉลยเป็นทางการ] Ex 3 (Statements in main program & Remaining Element):

```
myqueue = Queue(5)    # 1. ประกาศสร้าง Instance ใหม่เนื่องจากไม่มีตัวแปรมาก่อน (No queue before)
for i in range(1, 6):  # 2. Enqueue ค่า 1, 2, 3, 4, 5 ตามลำดับ เข้าสู่ myqueue
    myqueue.enqueue(i)
for _ in range(3):     # 3. Dequeue ข้อมูลออก 3 ครั้ง (ดึงค่า 1, 2, 3 ออกไปตามลำดับ FIFO)
    myqueue.dequeue()
-> The element that remain on myqueue: [4, 5] (Front ชีท 4, Back ชีท 5 ขนาดคงเหลือ = 2)
```

Is Different from

For example 3.1: if there is **the exist** queue's instance variable **myqueue** with size=5 contains the numbers [10, 20, 30, 40] ...

[ข้อควรระวังสำคัญ] สรุปหลักการ & ค่าเตือนจุดได้ 0 คะแนน (Why 3.1 is Different from 3):

1. ใน Example 3.1: มีอินสแตนซ์ **myqueue** เดิมอยู่แล้ว และมีข้อมูลบรรจุอยู่แล้วคือ [10, 20, 30, 40]
2. [ค่าเตือน 0 คะแนน] ข้อควรระวังสูงสุดในห้องสอบ (จุดตัด 0 คะแนน):
ห้ามเขียนคำสั่ง `myqueue = Queue(5)` ซ้ำในข้อ 3.1 เด็ดขาด!
เพราะการสร้างอินสแตนซ์ใหม่ จะไป Overwrite หัวตัวแปรเดิม ทำให้ข้อมูล [10, 20, 30, 40] หายไปทั้งหมด
3. แนวทางการตอบที่ถูกต้อง:
ต้องเรียกใช้เมธอดบนตัวแปร **myqueue** เดิมทันที เช่น `myqueue.dequeue()` หรือ `myqueue.enqueue(...)` โดยไม่ประกาศคลาสใหม่

[เปรียบเทียบโค้ด] เปรียบเทียบการทำงานระหว่าง Example 3 vs Example 3.1:

* แบบที่ 1 (Example 3 - No queue before):

```
myqueue = Queue(5) # จำเป็นต้องประกาศเพื่อสร้างออบเจกต์ขึ้นมาใหม่ในหน่วยความจำ
myqueue.enqueue(1); myqueue.enqueue(2); ...
```

* แบบที่ 2 (Example 3.1 - Exist queue [10, 20, 30, 40]):

ห้ามเขียน `myqueue = Queue(5)` เด็ดขาด (มิฉะนั้นจะกลายเป็น Queue ว่าง ได้ 0 คะแนน)

ให้เขียนดำเนินการต่อจาก Queue เดิมทันที เช่น:

```
myqueue.dequeue() # ดึง 10 ออก (FIFO หัวคิว) -> เหลือ [20, 30, 40]
myqueue.enqueue(99) # ใส่ 99 เข้าท้ายคิว -> กลายเป็น [20, 30, 40, 99]
```

[กฎข้อสอบ] กฎการทำข้อสอบเรื่อง Queue (FIFO - First In First Out):

- * การเข้าถึงข้อมูล: ข้อมูลตัวแรกที่ Enqueue เข้าไป จะเป็นตัวแรกที่ถูก Dequeue ออกมาเสมอ (Front -> Back)
- * การเขียนคำตอบในสไลด์/ใบงาน: ให้ระบุภาพ Queue จากหัวคิวไปยังท้ายคิว พร้อมระบุพอยน์เตอร์ Front และ Back
- * Circular Queue: เมื่อพอยน์เตอร์เดินถึงขอบอาร์เรย์ ($\text{Index} = \text{Capacity} - 1$) จะวนกลับมาที่ช่อง 0 ด้วย Modulo %